PROJECT MAP · ~/WORKSPACE/WORK/PROJECT/ANTICOUNTERFEITMED

Anticounterfeit Med

Medicine packages carry a hidden QR under a scratch coating. The manufacturer signs each unit with ML-DSA-65. A consumer scans it in the app and records a first verification, which can happen only once. This page maps every part of the repository and how the parts connect in production.

`Django 5.2 + DRF` `PostgreSQL 17` `Celery + Redis` `Next.js 15` `Flutter · Android` `ML-DSA-65 (FIPS 204)` `Firebase App Check`

System Label QR Unit lifecycle Repository Live deployment Commands

HOW IT FITS TOGETHER

System, layer by layer

Requests enter through the host's nginx, which handles TLS, then reach Docker containers bound to the loopback interface only. The signer sits on an internal network that only the API can reach.

People
who touches the system

Consumer

`consumer-app/ · Flutter`

Scratches the coating, scans, sees signed product info, confirms a first verification.

→ `api.pqc.anticounterfeitmed.com`

Staff

`browser · staff-web/`

Platform admins and manufacturers: batches, labels, print scans, activation, reports, audit.

→ `pqc.anticounterfeitmed.com/staff`

Any phone camera

`ordinary QR scanner`

Reads only the public URL on a label. Opens an information page and changes nothing.

→ `anticounterfeitmed.com`

Entry points
host nginx, TLS

api.pqc.anticounterfeitmed.com

`→ 127.0.0.1:8001`

The API the mobile app talks to. No cookies involved.

pqc.anticounterfeitmed.com

`infra/nginx/pqc-hub.conf`

`/staff` → portal :3100
`/v1/` → API :8001
`/` → hub :8080 (landing + APK)

anticounterfeitmed.com

`separate repo · medicine-pqc`

Public project website. This repo's `landing/` is the minimal static fallback page.

Services

Docker, infra/docker-compose.prod.yml

api
`backend/ · Django + DRF`
The only writer to PostgreSQL. All `/v1` routes: consumer, staff, admin.

worker · beat
`backend/ · Celery`
Background activation jobs, outbox, scheduled tasks.

portal
`staff-web/ · Next.js`
The staff portal, served under `/staff`.

hub
`hub/ · static`
Entry page with "Staff portal" / "Consumer app", and the APK download.

signer
`signer/ · medsigner`
Holds the private ML-DSA keys. No database access. Reachable only from api.

Data
state and queues

postgres
`PostgreSQL 17`
The only authority. A partial unique index enforces one first redemption per unit.

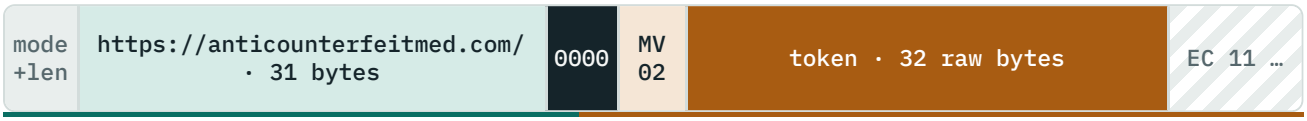
redis
`Redis`
Celery broker and cache only. Never gates a redemption.

Shared code: `libs/medcrypto` (Python crypto used by backend and signer) and `crypto-vectors/` (golden files that the Python, Dart and TypeScript implementations must all agree on).

OWNER DECISION D21 · 2026-09-22

What a label QR carries

One symbol, two readings. Every scanner stops at the terminator and sees only the public URL. The token rides after it, where only a raw-codeword reader finds it: ZXing in the app and in the portal's camera. ML Kit's `rawBytes` also stops at the text, which is why the app needs ZXing. That was the cause of the "not one of our codes" bug in the old APK.



Fixed shape

Always QR version 6 at error correction Q: 76 data codewords, same printed size on every label.

One layout, three parsers

`libs/medcrypto/medcrypto/labels.py` is the reference. `consumer-app/lib/core/scanned_url.dart` and `staff-web/lib/labelSymbol.ts` must match it on `crypto-vectors/label/`.

Hidden, not secret

Any raw-codeword decoder can read the token, and a photocopy still scans. First redemption stays the control.

DOCS/04-DOMAIN-MODEL.MD

Life of one medicine unit

`CREATED` → `PRINTED` → `QC_PASSED` → `COVERED` → `ACTIVE` → `REDEEMED`

Before ACTIVE: any state can go to `VOID` (cancelled, QC failure, damaged). `PRINTED` can be observed evidence via the print-line scan; QC pass and coating are always manufacturer-asserted.

REDEEMED is permanent: exactly one first consumer confirmation. Recalls and other restrictions are separate fields, not states.

ACTIVE: the manufacturer approves the batch and the signer issues a signed activation credential for each unit.

MONOREPO

Repository structure

Generated and dependency folders (`node_modules`, `.venv`, `build`, `migrations`, platform folders) are left out.

DEPLOYS runs in production SHARED used by several parts LOCAL gitignored

`Medicine_Verification_Implementation_Plan.md`	The authoritative requirements document. Wins any conflict with `docs/`.
`CLAUDE.md`	Invariants and working agreements: tokens stored only as SHA-256, REDEEMED is permanent, never claim "genuine".
`Makefile`	Every routine task: dev stack, tests, crypto vectors, app builds, label sheets, backups.
`README.md · pytest.ini · requirements-dev.txt`	Overview, test config, dev dependencies.
backend/	Django 5.2 + DRF. One backend for every role, and the only writer to PostgreSQL. DEPLOYS
config/	Project settings and wiring.
`settings.py`	Django settings, including `APP_CHECK_MODE`.
`urls.py`	Every `/v1` route: consumer, staff, admin, trust manifest.
`celery.py · wsgi.py · asgi.py`	Worker and server entry points.
apps/	Nine apps. They reach each other through service functions, never another app's models.
organizations/	Organizations, staff memberships, roles, MFA (`mfa.py`), permissions, `create_staff` command.
catalog/	Products, batches, reference images.
serialization/	Token generation, print jobs, label export (`export_store.py`). Issues each label's data codewords.
qc/	Printing, QC and coating records, and the print-line scan `POST /v1/staff/print-scans` (rate-limited).
activation/	Batch approval, signing orchestration, activation jobs.
verification/	Consumer prepare / confirm / status, consumer sessions, App Check attestation, signed envelopes, Celery tasks, throttling.
trust/	Signing keys and the root-signed trust manifest. Asks the signer to sign.
reports/	Consumer concern reports, investigation cases.

audit/	Append-only audit events, written in the same transaction as each change.
tests/	18 files against real PostgreSQL: race and idempotency, crypto vectors, label symbol, print scan, isolation, token-leak checks.
signer/	Isolated ML-DSA signing service. Holds private keys, has no database, and only the API can reach it. DEPLOYS
medsigner/keystore.py	Loads and guards private key material.
medsigner/server.py · service.py	The internal HTTP endpoint and the signing logic behind it.
Dockerfile · pyproject.toml	Container and package definition.
libs/medcrypto/	Python crypto used by both backend and signer. SHARED
labels.py	The label QR layout and its reference parser.
tokens.py	Token generation and format (43-char Base64url, 32 bytes).
signing.py · keys.py · contexts.py	ML-DSA-65 signing, key handling, and the required context strings.
records.py · canonical.py	Signed record shapes and their canonical byte encoding.
staff-web/	Next.js 15 staff portal, served at pqc.anticounterfeitmed.com/staff. DEPLOYS
app/login/	Sign-in with second factor.
app/(portal)/	Pages: batches, products, organizations, staff, reports, audit, settings.
components/LabelSheet.tsx	Lays out printable label sheets.
components/PrintScanner.tsx	Camera print-line scan that records PRINTED.
lib/labelSymbol.ts	Draws the QR from the backend's codewords (version 6-Q) and parses them back.
lib/readLabel.ts · scanned.ts · api.ts	Camera label reading, scan parsing, API client.
lib/vendor/qrcodegen.ts	Vendored QR encoder that accepts raw data codewords.
consumer-app/	Flutter, Android first. Distributed as anticounterfeitmed.apk from the hub.
lib/main.dart · app_state.dart	Start-up. BACKEND_BASE_URL and ROOT_PUBLIC_KEY are set at build time.
lib/screens/	scan, package, result, report, history.
lib/core/label_reader.dart	Reads every data codeword from a camera frame with ZXing.
lib/core/scanned_url.dart	Strict label parser. Anything else is "not one of our codes".
lib/core/outcomes.dart	Every verification outcome the app can show.
lib/crypto/	On-device ML-DSA verifier and a start-up self-check.

lib/data/	API client, Firebase App Check attestation, session store, trust store.
lib/l10n/strings.dart	Fixed consumer wording. A patient-safety requirement, not free copy.
assets/crypto-vectors/	Bundled vectors so the device can check its own verifier.
test · integration_test/	Unit tests, including the label vectors, and on-device tests.
crypto-vectors/	Golden files every implementation must match. SHARED
activation · status · negative/	Signed credentials that must verify, and tampered ones that must fail.
label/	Label symbols, valid and invalid, as JSON, PNG and a camera-style JPEG.
generate.py · generate_labels.py · verify.py · crosscheck_openssl.sh	Regenerate, verify, and cross-check against the system OpenSSL.
hub/	The pqc.anticounterfeitmed.com entry page and APK download. DEPLOYS
index.html · app.html	"Staff portal" / "Consumer app" chooser, and the app download page.
anticounterfeitmed.apk	Published app, copied here by make publish-app. LOCAL
landing/	Static public page with no JavaScript, strict CSP (csp.txt), its own nginx.conf.
index.html · nginx.conf · csp.txt	Page, server config, and CSP hashes (make landing-csp).
infra/	Deployment: Compose files, nginx, keys, backups.
docker-compose.prod.yml	postgres, redis, signer, api, worker, beat, hub, portal, proxy.
docker-compose.vps.yml	Live override: no proxy, services bound to loopback behind the host's nginx.
docker-compose.dev.yml	Local PostgreSQL and Redis.
nginx/	pqc-hub.conf (live routing) and conf.d/site.conf.
deploy.sh · provision-keys.sh	Deploy to the VPS; create signing keys on the server.
backup.sh · restore-drill.sh	Encrypted database backup and a restore rehearsal.
.env.deploy · .env	Deploy target and server settings. LOCAL
scripts/	Build and safety tooling.
publish_app.sh	Builds the app against prod (reads the root key over SSH); make app or make publish-app.

check_no_token_leak.sh · _filter_token_candidates.py	Fail if anything that looks like a raw token is tracked.
build_landing_csp.py · build_brand_assets.py	Landing CSP hashes; sized logo copies.
verify_restored_credentials.py · stop_dev_servers.sh	Restore-drill check; stop stray local servers.
docs/	The buildable plan, 00–17. Start at `00-index.md`.
01 scope · 02 architecture · 03 tech stack	What is in and out of scope, module boundaries, stack choices.
04 domain model · 05 API contract · 06 crypto design	States and entities, endpoints, signatures and keys.
07 security · 08 concurrency · 09 consumer app	Threats and privacy, locking order, fixed consumer wording.
10 portal & landing · 11 testing · 12 environments	Staff and public pages, acceptance gates, DevOps.
13 spikes · 14 roadmap · 15 open decisions · 16 glossary · 17 running locally	Plus `evidence/` for measured results.
spikes/	Week-0 experiments that settled early risks.
s1-qr/	Printed label tests: size, error correction, scannability.
s2-dart/	Proof that ML-DSA verification works in Dart.
assets/brand/	Project mark in every size.
dist/	Build outputs: APK and Play bundle. LOCAL
.keys/	Local development ML-DSA key seeds. LOCAL

VPS 187.53.136.115 · CHECKED 2026-09-23

Live deployment

One VPS shared with other projects. This project's containers are prefixed `infra-`. The host's nginx terminates TLS and proxies to them.

CONTAINER	BUILT FROM	REACHED AT	NOTES
infra-api-1	backend/	127.0.0.1:8001 ← api.pqc..., pqc.../v1/	APP_CHECK_MODE=accept-any for the pilot build

CONTAINER	BUILT FROM	REACHED AT	NOTES
`infra-worker-1` · `infra-beat-1`	`backend/`	internal	Celery worker and scheduler
`infra-portal-1`	`staff-web/`	`127.0.0.1:3100` ← pqc.../staff	Staff portal
`infra-hub-1`	`hub/`	`127.0.0.1:8080` ← pqc.../	Chooser page and APK
`infra-signer-1`	`signer/`	internal network only	Private keys; no database
`infra-postgres-1` · `infra-redis-1`	images	internal	Authority and broker

MAKEFILE

Commands you'll use most

`make up && make migrate && make seed`
Start PostgreSQL and Redis, migrate, add demo data.

`make serve`
Backend at 127.0.0.1:8000. Portal: **`cd staff-web && npm run dev`**.

`make check`
Everything CI runs: tests, vectors, leak check.

`make test`
Backend suite against real PostgreSQL.

`make app-test`
.
`make portal-test`
App and portal tests, label vectors included.

`make app`
Build the app against prod into **`dist/`**. Publishes nothing.

`make publish-app`
Build and publish the APK on the hub.

`make labels`
Printable physical QR test sheet (80 labels).

`make staff`
Create or reset the three demo portal accounts.

Mapped from the repository at commit `f9b593f` and the live server on 2026-09-23. Requirements: `Medicine_Verification_Implementation_Plan.md`.